components in a Redux JSX component called a Provider. (You might see a certain common quality to the Redux names and the React internal names we've been using—the Redux names came first, for what it's worth.) The provider takes one prop argument, the store. Notably, the state of the store is now as it is after we dispatched our init action. The Provider is not named to differentiate it from other providers—the expectation is that Redux providers will not be nested, any combination of reducers will be handled when the store is created.

To use the Redux store within those child components, we need two hooks: useDispatch and useSelector, and the TypeScript safe versions of them we just defined, which are useAppDispatch and useAppSelector.

The useDispatch hook takes no arguments and returns a dispatch function, so the usage is typically const dispatch = useDispatch(). The returned dispatch function is used just like the other dispatch functions we've been discussing—you pass it a known action and it returns the new state. The useAppDispatch method is defined as useDispatch<AppDispatch>(), meaning that it's the same method, but we've tied it to the dynamic AppDispatch type. This doesn't make a difference for us yet, but in a little bit when we start doing asynchronous dispatch actions it will allow us to keep that compatible with TypeScript.

The useSelector hook lets us get to the state of the store directly, and the argument is maybe not quite what you'd expect. The argument to useSelector is a function. This argument function takes one argument—the state of the Redux store—and returns whatever the heck you want. So you could just get back the entire state with const state = useSelector(state => state). In practice, though, you typically limit the return value to just the parts of the state that the component needs, so const seatsInRow = useSelector<VenueState, number>(state => state.seatsInRow). The useAppSelector version just presets the VenueState as the argument of the method so we don't have to retype it everywhere.

The TypeScript generic types on useSelector are the type of the entire state and the type of the return value. Limiting the return values is useful because then Redux will only trigger a redraw of the component if the return value changes rather